CÓDIGOCLARO · RUTA KOTLIN

Tema 15 · Pruebas y arquitectura

Cuaderno de estudio para DAM/DAW · 10 lecciones

Qué vas a aprender

JUnit, pruebas unitarias, capas y separación de responsabilidades.

Este cuaderno trabaja pruebas y arquitectura en pasos cortos. La meta no es repetir palabras de memoria: es reconocer el problema, elegir una herramienta adecuada, explicar por qué funciona y comprobarla con datos distintos. Reserva una sesión para leer el modelo, otra para resolver el ejercicio y una tercera para intentar el laboratorio sin mirar la solución.

Resultados de aprendizaje

- Explicar con tus palabras qué problema resuelve pruebas y arquitectura y en qué parte de una aplicación DAM/DAW aparece.
- Leer el ejemplo, predecir el resultado y justificar cada decisión relevante.
- Resolver una práctica, comprobar casos normales y límite, y depurar un fallo con un método reproducible.

Cómo estudiar este tema

Ten a mano un editor compatible con la tecnología y copia los ejemplos en un archivo nuevo. Antes de ejecutar, escribe tu predicción. Si no coincide con la salida, no borres el intento: anota qué supuesto era incorrecto y prueba una variación mínima. Al terminar, explica el concepto a otra persona usando un ejemplo propio.

Punto de control inicial

Antes de avanzar, escribe dos ideas que ya conoces sobre este tema y una pregunta que todavía no sabes responder. Vuelve a ellas al final de la lección 10. Si cambias de respuesta, explica qué ejemplo o prueba hizo que cambiaras de opinión.

Lección 1 · El modelo mental

Idea central

Una prueba unitaria compara un resultado observable con el contrato esperado. kotlin.test ofrece anotaciones y assertions comunes que se conectan con un runner como JUnit según la configuración del proyecto. Una función de dominio pura es fácil de probar porque el mismo input produce el mismo resultado. Separar UI, dominio y acceso a datos reduce dependencias cruzadas; no es obligatorio crear capas para un ejercicio pequeño si no mejoran la claridad.

Para razonar sobre pruebas y arquitectura, separa cuatro preguntas: qué datos existen al empezar, qué regla se aplica, qué cambia durante el proceso y qué resultado debe observar quien usa el programa. Esta separación ayuda a no confundir la sintaxis con la finalidad. Una línea puede compilar y aun así resolver mal el problema si el dato representa otra cosa o falta un caso.

En Kotlin, ten presente este marco

El itinerario usa Kotlin/JVM: el compilador infiere muchos tipos, pero el programa sigue teniendo comprobación estática. val impide reasignar el nombre, aunque el objeto referenciado pueda ser mutable; var permite reasignar. Los tipos nullable se escriben con ? y obligan a tratar la ausencia. La biblioteca distingue interfaces de solo lectura de interfaces mutables.

Comprueba tu comprensión

- ¿Qué parte de la explicación define la regla principal de pruebas y arquitectura?
- ¿Qué dato puede cambiar y qué dato debería permanecer estable?
- ¿Qué observación concreta demostraría que tu programa cumple el objetivo?

Lección 2 · Vocabulario y reglas

Explicación técnica de pruebas y arquitectura

Una prueba unitaria compara un resultado observable con el contrato esperado. kotlin.test ofrece anotaciones y assertions comunes que se conectan con un runner como JUnit según la configuración del proyecto. Una función de dominio pura es fácil de probar porque el mismo input produce el mismo resultado. Separar UI, dominio y acceso a datos reduce dependencias cruzadas; no es obligatorio crear capas para un ejercicio pequeño si no mejoran la claridad.

Errores concretos que cambian el resultado

- Probar llamadas privadas internas en vez del contrato visible.
- Hacer que una prueba dependa de red, hora o estado compartido sin controlarlo.

Resultado del ejemplo

El runner descubre calculaDoble y la assertion pasa.

Lección 3 · Lectura guiada del ejemplo

Primero identifica la entrada, luego sigue el valor por el programa. No supongas que una asignación transforma el dato: a veces solo enlaza un nombre con el mismo objeto. Observa dónde empieza cada bloque, qué condición activa una rama y qué instrucción produce la salida. Si el ejemplo tiene varias líneas, ejecútalo por partes y añade temporalmente observaciones seguras para ver los valores intermedios.

Código · kotlin

```kotlin
import kotlin.test.Test
import kotlin.test.assertEquals

fun doble(numero: Int) = numero * 2

class DobleTest {
    @Test fun calculaDoble() {
        assertEquals(8, doble(4))
    }
}
```

Salida esperada

```
El runner descubre calculaDoble y la assertion pasa.
```

Registro de la traza

Momento	Qué anotar
Antes de ejecutar	Valores iniciales, sus tipos y qué operación esperas que ocurra.
Después de cada paso	Nombre afectado, valor nuevo y regla que produjo el cambio.
Al encontrar una decisión	Condición evaluada y camino que se ejecutó.
Al terminar	Salida real, diferencia respecto a tu predicción y causa.
Con una entrada distinta	Qué resultado cambia y qué parte del contrato permanece igual.

Traza en papel

- Antes de la primera instrucción: apunta los valores iniciales y sus tipos.
- Después de cada operación: anota solo los nombres que hayan cambiado y la razón.
- Al final: compara tu predicción con «El runner descubre calculaDoble y la assertion pasa.» y explica la diferencia si la hay.

Cambia un dato de entrada y vuelve a hacer la traza. Una buena explicación debe anticipar el nuevo resultado, no limitarse a repetir la salida original.

Lección 4 · Decidir cuándo usarlo

Una prueba útil hace visible una regla: prepara una entrada, ejecuta una unidad y compara el resultado con una expectativa. Incluye un caso habitual, uno límite y uno inválido cuando el contrato lo permita. Depurar consiste en reducir el problema, observar el valor real y comprobar una hipótesis; no en añadir cambios al azar hasta que desaparezca el síntoma.

Caso de uso

Una tienda aplica descuentos con una tasa acotada entre 0 y 1 y necesita que los cambios de la fórmula no alteren los resultados.

Compara antes de elegir

- ¿La solución más directa deja claro el propósito o requiere conocer demasiados detalles internos?

- ¿Qué coste aparece si crece el volumen de datos o se añade un nuevo caso?
- ¿Qué comprobación automatizada detectaría una modificación accidental?

Enfoques relacionados

- Usar kotlin.test con el runner ya configurado.
- Usar las assertions de JUnit directamente si el proyecto depende de JUnit.

No existe una alternativa mejor en todos los contextos. Elige una y escribe su ventaja y su límite para el tamaño, la versión y los requisitos del ejercicio. Si dos enfoques cumplen exactamente el mismo contrato y ambos son legibles, pueden considerarse válidos.

Lección 5 · Errores y diagnóstico

Fallos que conviene reconocer

- Probar llamadas privadas internas en vez del contrato visible. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Hacer que una prueba dependa de red, hora o estado compartido sin controlarlo. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.
- Crear interfaces y capas sin una razón que reduzca acoplamiento o facilite cambios. Para localizarlo, reduce el caso a la menor entrada que lo reproduce, observa el valor implicado y verifica la regla antes de cambiar el código.

Rutina de depuración

1. Describe el resultado esperado y el resultado real sin mezclar ambos. 2. Reproduce el fallo con una entrada fija. 3. Reduce el programa hasta dejar la primera operación que se comporta distinto. 4. Formula una hipótesis comprobable y cambia una sola cosa. 5. Añade una prueba que impida que el mismo fallo vuelva. Este orden también funciona cuando el error aparece en una API, una interfaz o un archivo.

Preguntas de revisión

- ¿El error está en los datos, en la regla o en cómo se presenta el resultado?
- ¿Qué caso límite no estaba cubierto por el ejemplo inicial?
- ¿La solución corrige la causa o solo oculta el síntoma?

Lección 6 · Práctica guiada

Situación de DAM/DAW

Una tienda aplica descuentos con una tasa acotada entre 0 y 1 y necesita que los cambios de la fórmula no alteren los resultados.

Enunciado

Implementa una función que aplique un porcentaje de descuento y añade tres pruebas. La tasa debe estar entre 0 y 1.

Convierte el enunciado en requisitos verificables

- Escribe qué entradas acepta la solución y qué debe devolver o mostrar.
- Enumera las reglas de negocio y separa las que son obligatorias de las decisiones de presentación.
- Antes de programar, completa la tabla de pruebas y predice cada resultado.

Casos que debe superar

Entrada	Resultado esperado
(100.0, 0.0)	100.0
(100.0, 0.25)	75.0
(50.0, 1.0)	0.0

Qué demuestra cada caso

Clasifica cada fila de la tabla como caso habitual, límite o variación importante. Si el enunciado no define qué hacer con una entrada fuera de contrato, no inventes un resultado silencioso: anota la ambigüedad y propone una regla antes de programar. En la práctica profesional, acordar esa regla evita que dos implementaciones parezcan correctas y se comporten de forma incompatible.

Plan de solución

- Divide la tarea en pasos pequeños que puedan comprobarse por separado.
- Implementa primero el caso habitual y después añade los límites y errores previstos.
- Ejecuta cada caso tras un cambio; no esperes al final para descubrir qué parte falló.

Lección 7 · Solución explicada

Una solución no se justifica solo porque la salida coincida en un ejemplo. Debe respetar el contrato para todas las entradas admitidas. Lee el código siguiente y vincula cada operación con una frase del enunciado. Si no puedes señalar esa relación, revisa el diseño antes de continuar.

Código · kotlin

```kotlin
import kotlin.test.Test
import kotlin.test.assertEquals
import kotlin.test.assertFailsWith

fun conDescuento(precio: Double, tasa: Double): Double {
    require(tasa in 0.0..1.0)
    return precio * (1 - tasa)
}

class DescuentoTest {
    @Test fun tasaCero() {
        assertEquals(100.0, conDescuento(100.0, 0.0))
    }

    @Test fun tasaParcial() {
        assertEquals(75.0, conDescuento(100.0, 0.25))
    }

    @Test fun tasaCompleta() {
        assertEquals(0.0, conDescuento(50.0, 1.0))
    }

    @Test fun rechazaTasaFueraDeRango() {
        assertFailsWith<IllegalArgumentException> { conDescuento(100.0, 1.1) }
    }
}
```

Criterio de aceptación: Se acepta redondeo explícito si el dominio define precisión monetaria; las pruebas deben cubrir las tres tasas del enunciado y el rango inválido.

Por qué se sigue este razonamiento

require comprueba que la tasa esté entre 0.0 y 1.0; después se multiplica el precio por 1 - tasa. Las tres pruebas verifican 100.0, 75.0 y 0.0 para las tasas 0.0, 0.25 y 1.0. La prueba adicional comprueba que una tasa fuera del rango lanza IllegalArgumentException.

Comprobación de corrección

Comprueba cada caso de la tabla y explica qué parte de la solución lo cubre. Después revisa el caso mínimo, el máximo razonable y una entrada inválida si el contrato la permite. Comprueba también los tipos de entrada y salida: una coincidencia textual puede ocultar un valor mal tipado o una conversión con pérdida.

Paso	Justificación que debes poder dar
Contrato	Qué valores acepta el ejercicio y qué resultado promete.
Regla	Qué operación transforma la entrada en la salida.
Pruebas	Qué caso comprueba lo normal y cuál descubre un límite.

Alternativas válidas

- Usar kotlin.test con el runner ya configurado.
- Usar las assertions de JUnit directamente si el proyecto depende de JUnit.

Al comparar soluciones, prioriza contrato correcto, legibilidad y pruebas. Una alternativa que cambia la representación interna puede ser válida; una que cambia silenciosamente el significado de la salida no lo es.

Lección 8 · Laboratorio de aplicación

Retoma el ejercicio de pruebas y arquitectura y amplíalo como una pequeña funcionalidad de una aplicación de gestión académica. Conserva el requisito original y añade una regla nueva que tenga sentido para los datos. Antes de modificar el código, escribe la regla en lenguaje natural, un ejemplo que la cumpla y otro que la incumpla.

Variación para transferir lo aprendido

Añade pruebas para tasa 0.0 y 0.25, más una aserción de que una tasa fuera del rango lanza IllegalArgumentException.

Desafíos progresivos

- Nivel 1: añade un caso límite que no aparecía en la tabla y registra el resultado esperado.
- Nivel 2: cambia la forma de recibir o recorrer los datos sin cambiar el contrato externo.
- Nivel 3: separa la regla principal de su entrada/salida y prueba ambas partes de forma independiente.

Criterios de entrega

- La solución conserva el resultado esperado del ejercicio original.
- El nuevo comportamiento queda cubierto por una prueba que falla antes del cambio.
- Los nombres permiten seguir el flujo sin comentarios que repitan la sintaxis.
- La entrada inválida o el error externo tiene una respuesta definida.

Prueba de regresión

Guarda los casos originales antes de añadir la regla nueva. Ejecuta la batería completa después de cada cambio, no solo el nuevo caso. Si una prueba deja de pasar, decide si encontraste un defecto o si cambió el contrato y hay que actualizar la expectativa con una explicación. Termina con una nota breve: qué cambió, qué comprobaste y qué límite sigue abierto.

Anota qué parte se puede reutilizar en otro módulo y qué parte depende del contexto de la aplicación. Esa distinción ayuda a decidir si merece la pena extraer una función o componente.

Lección 9 · Síntesis y fuentes

Resumen del tema

Has estudiado pruebas y arquitectura desde el modelo conceptual hasta una práctica comprobable. La secuencia de trabajo es: identificar entradas y contrato, elegir la operación adecuada, seguir el flujo, comprobar casos límite y conservar una prueba para la siguiente modificación. Aplica las reglas de Kotlin sin confundir una característica del lenguaje con una API concreta del entorno.

Lista de control antes de avanzar

- Puedo explicar el concepto con un ejemplo propio y predecir su salida.
- Puedo identificar al menos un caso límite y un error habitual.
- Puedo comparar dos enfoques y justificar mi elección.
- Puedo verificar una solución sin depender de mirar el resultado una sola vez.

Fuentes oficiales para ampliar

JetBrains / Kotlin — kotlin.test API · JUnit Team — JUnit 5 User Guide

Consulta la sección de la fuente que corresponda a la versión de tu proyecto. Este cuaderno resume conceptos para estudiar; cuando haya una diferencia de detalle, prevalece la documentación oficial del lenguaje o biblioteca.

Lección 10 · Repaso activo

Cierra el material y responde sin copiar frases. Si dudas, vuelve a la página indicada, comprueba la fuente primaria y reescribe la respuesta con tus palabras. La recuperación activa ayuda a detectar qué parte aún depende de memorizar un ejemplo concreto.

Preguntas

- ¿Qué problema aborda pruebas y arquitectura y qué dato necesita?
- ¿Qué diferencia hay entre una solución que funciona en el ejemplo y una que cumple el contrato?
- ¿Qué error frecuente te parece más probable y cómo lo reproducirías?
- ¿Qué alternativa elegirías en un proyecto pequeño y por qué?

Respuestas de comprobación

- La idea clave se resume en esta explicación: Una prueba unitaria compara un resultado observable con el contrato esperado. kotlin.test ofrece anotaciones y assertions comunes que se conectan con un runner como JUnit según la configuración del proyecto. Una función de dominio pura es fácil de probar porque el mismo input produce el mismo resultado. Separar UI, dominio y acceso a datos reduce dependencias cruzadas; no es obligatorio crear capas para un ejercicio pequeño si no mejoran la claridad.
- Una fuente inicial de fallo descrita en el tema es: Probar llamadas privadas internas en vez del contrato visible.
- Una alternativa es válida si mantiene las entradas, salidas y reglas acordadas y supera las mismas pruebas.